Sudoku

NF-1400 Object-oriented programming - Assignment 1

In this assignment, you are going to write a program that finds solutions to the popular puzzle known as Sudoku.

Prerequisites

To solve this assignment you should have a basic understanding of the following concepts

- String and integer data types
- Collections such as lists, tuples, dicts, and sets
- Classes and inheritance
- Reading from a text file
- Code testing (with pytest)
- Recursion

Learning outcomes

After this assignment you will be more familiar with and proficient in

- Classes, objects and inheritance
- Designing systems with multiple classes and many objects
- Linking objects via references
- Searching for solutions using depth-first search

“Standard” Sudoku

The most well-known version of a sudoku board has 9x9 squares, containing the digits 1-9. Before the puzzle is solved, only some squares are filled in:

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Example 9x9 Sudoku puzzle. By user Auguel, Wikimedia Commons.

The goal of the puzzle is to fill in all the squares so that each row, each column, and each “box” (3x3, separated by bold lines) contain all the digits 1-9. Note that this means that no digits are repeated within a row/column/box. The finished version of the puzzle above looks like this:

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

Solution to example 9x9 Sudoku puzzle. By user Auguel, Wikimedia Commons.

Sudoku puzzles can be designed to be easy or difficult. Generally, the more of the squares that are filled in, the easier the puzzle. Also, in some cases, the combinations of pre-filled values in a square's row, column and box allow only a single value, so that the value can be filled in immediately. However, this is not always the case.

Sudoku variations

Many variations of Sudoku have been made. The 4x4 and 6x6 boards are common and are also popular because they are a bit easier.

2			1
1		2	
	1	4	
	2	1	

4x4 example board

			4		2
	2			5	6
6			3	1	5
		5		2	4
2	3	4			
5	6		2		3

6x6 example board

Note that the boxes for 6x6 Sudoku puzzles are not "square", but 2x3 rectangles.

We are going to take advantage of the concept of *inheritance* in object-oriented programming. You will first define a generic Sudoku board that can have any size, and later you will create specialized versions of this for 9x9, 6x6 and 4x4 boards.

Example boards

The assignment includes a number of example sudoku boards, and a SudokuReader class for reading the boards. The sudoku boards include solutions, so that you can check if your solver works as it should.

Here's an example 4x4 board with its solution, as it appears in the text file. Blank squares are marked with a period.

```
1.23
2..4
.1.2
....

1423
2314
3142
4231
```

The reader converts a board into a nested list of integer numbers, with blank spaces represented as zeros:

```
puzzle   =  [[1,0,2,3], [2,0,0,4], [0,1,0,2], [0,0,0,0]]
solution =  [[1,4,2,3], [2,3,1,4], [3,1,4,2], [4,2,3,1]]
```

Starter code

As a starting point for this assignment you get a code "skeleton" that defines the following classes:

- **Sudoku**: An abstract class for a sudoku board, with methods for initialization, visualization of the board, and for solving a puzzle
- **Sudoku_4x4, Sudoku_6x6, Sudoku_9x9:** Subclasses of Sudoku that are adapted to different sized boards
- **Square**: A class representing a single square on the board, with a method for calculating possible valid values (for blank squares).
- **Element**: An abstract class representing a generic part of the board containing multiple squares
- **Row, Column, Box**: Subclasses of Element

The classes are all empty - it is up to you to implement them.

“Brute force” solving algorithm

The simplest Sudoku solving algorithm is the “brute force” method. The idea is basically to systematically try every combination of values in the blank squares until you find a valid solution. The algorithm can be described as follows (in “pseudocode”)

```
solve(board):
    if the board is solved (all squares have valid values),
        return the board
    else,
        get the next blank square on the board
        for each valid value that the square can have,
            assign the value to the square
            try to solve the rest of the board
            if a solution was found,
                return the solution

        (if this point is reached, none of the possible values worked)
        reset the square so that it is blank again
        return indicator of "no solution found"
```

This is an example of “depth-first search”, where we follow possible solutions as far as they go, and if we reach a “dead end”, we go a step back (we “backtrack”) and follow alternative paths. Note that the algorithm is *recursive*, meaning that it calls itself.

There are numerous other algorithms that you can experiment with, which will also be considerably quicker than the brute-force method. We do, however, recommend that you write a working brute-force solution first, just to make sure you have something you can hand in in case your more advanced algorithm doesn’t work as intended.

See also: “Backtracking” Sudoku algorithm described on Wikipedia.

Requirements to pass

The main point of the assignment is demonstrating that you know how to create classes and subclasses (via inheritance), and that these classes correspond to properties and functionality related to Sudoku. It is not required that your solution solves *all* Sudoku boards, but your algorithm should at least be able to solve some of the easier boards. Ask lecturers, teaching assistants or fellow students if you get stuck.

Use of AI

This assignment allows use of AI up to level 2 as defined by UiT. The following use of AI is allowed:

- Discussing ideas
- Drafting the structure (i.e. “headlines”) of code and report
- Translating between languages
- Spell checking and “cleaning” text (that you have written yourself)

If you do use AI, you are required to deliver:

- A declaration of AI use, where you describe how you’ve used AI. See also: Example of declaration of AI use.
- A “reflection note” where you reflect (i.e. discuss with yourself) on the advantages and disadvantages of using AI for the learning outcomes of this assignment.

For additional information about using AI at UiT, see the official UIT guidelines on AI.

Additional information

You are free to go beyond the assignment and write extensions such as a GUI to display sudoku boards, optimizing algorithms for speed, etc. We want you to have fun with programming and embrace creativity! You can of course also add additional functions, methods, classes and variables wherever you find it useful for your solution.

This is an individual assignment, and thus everything you hand in must be your own work unless you clearly specify that parts are quotes or references. Read more about the rules regarding mandatory assignments and exams on our Canvas page.